



Multi-Agent & Kết Nối Hệ Thống

AICB-P1 · Ngày 9 · MCP, A2A & LangGraph

Tên Giảng Viên

VinUniversity · Phase 1 · Tuần 2 · 2026

HÃY SUY NGHĨ...

“Bạn có 1 agent rất giỏi. Nhưng bài toán đã quá lớn cho 1 agent. Làm thế nào để hệ thống vẫn rõ vai trò, dễ kiểm soát, và dễ mở rộng?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

D1
Prompt

D2
RAG

D3
Eval

D4
Tools

D5
Memory

D6
Safety

D7
Prod.

D8
Adv. RAG

D9
Multi-Agent

Day 08 đã dạy cách **lấy đúng thông tin**. Day 09 hỏi câu tiếp theo: khi bài toán lớn hơn một agent, ta **tổ chức hệ thống** như thế nào?

1. Giới hạn của single-agent
2. Mental model: tư duy hệ thống
3. Multi-agent patterns
4. Supervisor-worker deep dive
5. MCP – chuẩn kết nối tool
6. A2A – giao tiếp giữa agents
7. Orchestration với LangGraph
8. Observability & debugging
9. Cost, latency & reliability
10. Kế hoạch học tập
11. Lab 9 + deliverable

một agent → nhiều agent → kết nối ra ngoài (MCP/A2A) → điều phối (LangGraph)
→ quan sát được (trace).

- Giải thích được vì sao **single-agent** bắt đầu quá tải khi bài toán cần nhiều vai trò và nhiều nguồn lực
- Phân biệt được các pattern **supervisor-worker**, **pipeline**, **debate**, và **hierarchical** – và biết chọn đúng pattern
- Hiểu **MCP** là chuẩn nối agent với tool / service bên ngoài, và **A2A** là cách agents giao việc cho nhau với message contract rõ
- Dùng **LangGraph** để hình dung graph, state, và conditional routing trong hệ multi-agent
- Thiết kế được **trace & observability** để debug và cải thiện hệ thống
- Nâng cấp artifact Day 08 thành hệ thống **Supervisor + Workers** có trace rõ ràng

Bản nâng cấp từ Day 08 gồm supervisor, 2–3 workers, 1 kết nối tool qua MCP, và trace log cho toàn bộ luồng phối hợp

- 1 supervisor nhận task, route đúng worker, và tổng hợp kết quả
- 2–3 worker chuyên vai trò như retrieval, tool use, synthesis
- 1 kết nối external capability qua **MCP**
- 1 trace để đọc để giải thích agent nào đã làm gì và khi nào

Từ Single-Agent Sang Multi-Agent

Day 08 giúp agent biết retrieve và trả lời grounded; Day 09 trả lời câu hỏi khi nào một agent không còn đủ để gánh toàn bộ bài toán

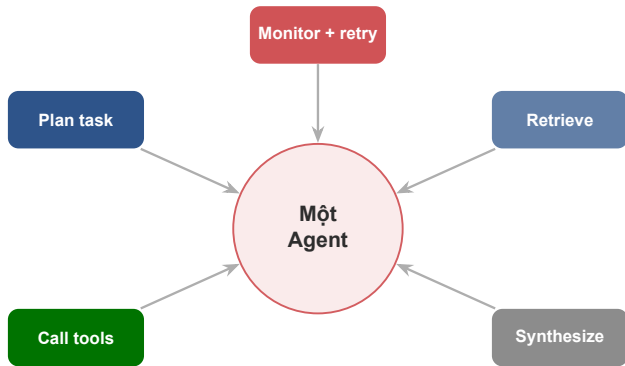
Day 08 đã làm được

- retrieve đúng tài liệu hơn
- rerank hoặc lọc context tốt hơn
- generate câu trả lời grounded hơn

Nhưng khi hệ thống lớn lên

- phải phân tích task trước khi retrieve
- phải gọi thêm tool ngoài
- phải chia việc và tổng hợp nhiều kết quả
- phải theo dõi trace để debug

Day 09 không phủ định Day 08. Nó là bước tiếp theo: **biến một agent có RAG thành một hệ thống có vai trò, có phối hợp, và có điểm mở rộng rõ ràng.**



Một nơi phải làm quá nhiều việc sẽ khó tối ưu, khó debug, và khó scale.

Lưu ý: Câu hỏi đúng không còn là “agent có đủ thông minh không?” mà là “ta có đang ép một agent gánh quá nhiều vai trò không?”

1. Context bottleneck

Một agent phải giữ quá nhiều mục tiêu, tool outputs, evidence, và state trong cùng một lần suy luận. Context window có giới hạn cứng.

2. Specialization trade-off

Agent càng ôm nhiều vai, prompt càng dài và khó ổn định. Giỏi đều mọi thứ thường đồng nghĩa với không thật sự giỏi vai nào.

3. Parallelism hạn chế

Một agent thường chạy tuần tự. Khi có nhiều việc độc lập, hệ thống vẫn phải chờ từng bước nối nhau – tăng latency không cần thiết.

4. Reliability yếu

Nếu agent chọn sai tool hoặc hiểu sai task ở đầu luồng, toàn bộ hệ thống dễ đi lệch theo. Không có isolation để khoanh vùng lỗi.

Kịch bản thực tế

- Giả sử context window **24k tokens**; task: phân tích hợp đồng 80 trang + tra luật
- Tool call trả về 12,000 tokens
- Chat history đã chiếm 6,000 tokens
- Prompt gốc: 3,000 tokens
- Còn lại cho reasoning: $\approx 3,000$ tokens

Dấu hiệu nhận biết

- câu trả lời thiếu thông tin ở giữa document
- agent lặp lại bước đã làm
- reasoning ngắn bất thường ở bước cuối
- tool call với empty context

Lưu ý: Agent bắt đầu “quên” phần đầu tài liệu khi xử lý phần cuối – đây là **lost-in-the-middle problem**.

Nguyên tắc nền (Anthropic, 2024)

- “Chỉ thêm hệ agentic nhiều bước **khi giải pháp đơn giản hơn đã không đủ.**”
- Tối ưu một LLM call với retrieval + ví dụ thường đã giải quyết phần lớn bài toán.
- Agentic tự chủ $\Rightarrow$ chi phí cao hơn và lỗi dễ **cộng dồn.**

Cái giá phải trả

Nhiều agent = nhiều LLM call = nhiều token. Con số cụ thể nằm ở phần **Cost & Reliability** – multi-agent có thể tốn **hơn cả chục lần** token so với một lần chat.

Lưu ý: Multi-agent không phải là “bản nâng cấp miễn phí”. Nó là một **đánh đổi** bạn chọn khi bài toán thực sự cần – không phải vì nó nghe “ngầu”.

- ✓ **Task có nhiều bước vai trò khác nhau:** plan, retrieve, tool use, tổng hợp
- ✓ **Có thể chia việc độc lập:** 2 worker làm song song vẫn hợp lý
- ✓ **Cần debug rõ ai làm sai:** route sai, retrieve sai, hay synthesis sai
- ✓ **Cần mở rộng dần:** thêm 1 worker mới mà không viết lại cả prompt gốc
- ✓ **Context window một agent không đủ:** tài liệu lớn, nhiều tool output
- ☐ **Đừng dùng multi-agent chỉ vì thấy “ngầu”.** Nếu 1 workflow đơn giản đã đủ, giữ đơn giản sẽ dễ và ổn định hơn

Mental Model: Tư Duy Hệ Thống

Trước khi chọn pattern hay tool, cần hình thành tư duy đúng về cách chia trách nhiệm trong một hệ thống phức tạp

Tư duy cũ

- Làm thế nào để agent thông minh hơn?
- Prompt nào khiến agent làm được nhiều hơn?
- Thêm nhiều tool cho một agent để đủ sức?

Tư duy hệ thống

- Task này gồm bao nhiêu loại trách nhiệm khác nhau?
- Ai cần biết gì, khi nào?
- Lỗi cần được khoanh vùng ở đâu?
- Điểm nào cần human oversight?

Lưu ý: Tư duy cũ dẫn đến “**god agent**” – một agent làm hết nhưng không ai hiểu nó đang làm gì. Tư duy hệ thống cho ta vai trò rõ, để test từng phần, để cải thiện dần.

Task nào cần reasoning?
Task nào cần data fetch-
ing? Task nào chỉ cần for-
mat?

Agent nào cần biết gì? Ai
cần đầu ra của ai trước
tiên?

Thiết kế để lỗi tại một
worker không làm hỏng
toàn bộ hệ thống.

Thiết kế tốt giúp lỗi có **địa chỉ rõ ràng** thay vì lan ra cả hệ thống.

Anthropic (2025): multi-agent thắng

Hệ research nhiều agent (lead + 3–5 subagent song song) mạnh ở việc **đọc rộng**: nhiều hướng độc lập, vượt quá một context window.

Cognition (2025): cẩn thận

Subagent song song dễ **ra quyết định ngầm mâu thuẫn** khi cùng **ghi** vào một sản phẩm chung (ví dụ: 1 subagent vẽ nền kiểu Mario, subagent kia làm con chim Flappy Bird) – nên ưu tiên một luồng liên tục.

Song song hoá việc đọc, tuần tự hoá việc ghi. Nếu các agent phải thống nhất trên một artifact chung → ưu tiên một luồng; nếu chúng khám phá các hướng độc lập → chạy song song.

Multi-Agent Patterns

Có nhiều cách chia hệ thống thành nhiều agent; điều quan trọng là chọn pattern giúp giải quyết đúng loại phức tạp, không tạo thêm phức tạp giả

03

1 supervisor điều phối nhiều worker chuyên biệt.

Mạnh ở: routing rõ, dễ kiểm soát, dễ trace

Agent A xong rồi mới chuyển output cho B.

Mạnh ở: flow ổn định, tuyến tính, dễ test

Nhiều agent cùng giải một bài toán rồi vote / synthesize.

Mạnh ở: phản biện, giảm blind spot

Hierarchical

Supervisor lồng supervisor cho nhiều tầng.

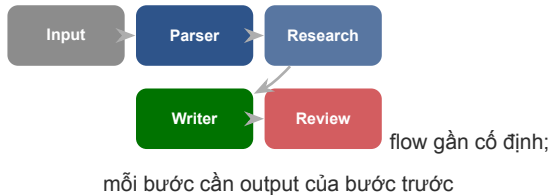
Mạnh ở: mở rộng tốt ở enterprise scale

Chúng ta đi sâu vào **supervisor-worker** vì đây là pattern dễ dạy, dễ build lab, và dễ nổi trực tiếp từ artifact Day 08.

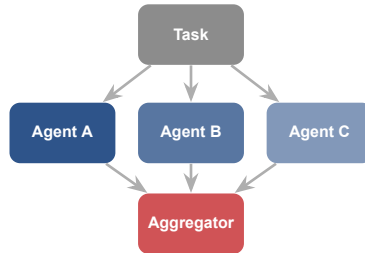
Pattern	Dùng khi nào	Điểm mạnh	Cảnh báo
Supervisor-worker	Task cần route tới đúng vai trò	dễ kiểm soát, dễ trace	supervisor có thể thành bottleneck nếu làm quá nhiều
Pipeline	Các bước gần như cố định	dễ hiểu, dễ test theo bước	kém linh hoạt khi flow đổi động
Debate	Cần nhiều góc nhìn cho cùng một bài toán	giảm blind spot, tăng phản biện	tốn cost và khó tổng hợp
Hierarchical	Nhiều nhóm task và nhiều tầng quản trị	mở rộng tốt ở quy mô lớn	thiết kế và debugging phức tạp

Đừng để 4 pattern ngang nhau trong đầu người học

Pipeline



Debate



nhều góc nhìn hợp
lệ; rủi ro sai cao; cần kiểm tra chéo

Pattern	Ý tưởng	Multi-actor?
Prompt chaining	chia task thành chuỗi bước, output bước này là input bước sau	1 LLM
Routing	phân loại input rồi đưa tới nhánh xử lý chuyên biệt	1 LLM
Parallelization	sectioning (chia việc độc lập) hoặc voting (chạy nhiều lần)	có
Orchestrator-workers	1 LLM trung tâm chia việc, giao worker, rồi tổng hợp – = supervisor-worker	có
Evaluator-optimizer	một LLM tạo, một LLM đánh giá & phản hồi trong vòng lặp	1 LLM

Nguồn: Anthropic, “Building Effective Agents” (2024) – framing được cộng đồng dùng rộng rãi

Lý do sự phạm

- học viên dễ nhìn ra vai trò
- dễ nối với use case thật
- dễ giải thích logic route
- dễ nâng cấp từ artifact Day 08

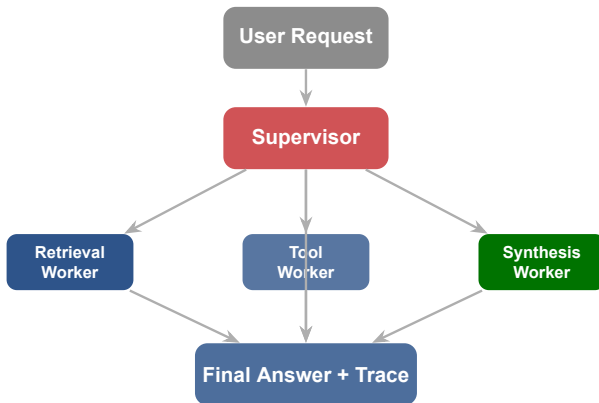
Lý do triển khai

- bắt đầu từ 2–3 worker là đủ
- dễ cắm thêm MCP tool worker
- trace và testing rõ hơn
- supervisor thường chỉ cần một LLM call nhỏ

Lưu ý: Nếu Day 09 ôm nhiều pattern ngang nhau, người học sẽ nhớ tên pattern nhưng không biết ngày mai nên build theo pattern nào.

Supervisor-Worker: Deep Dive

Thay vì ép một agent làm hết, ta cho một supervisor phân việc và nhiều worker làm phần việc hẹp, dễ kiểm soát hơn



Supervisor không cần “thông minh hơn tất cả”. Vai trò chính là **chia việc đúng, gọi đúng worker, và gom đầu ra thành kết quả mạch lạc.**

Supervisor

- phân tích yêu cầu ban đầu
- quyết định worker nào nên tham gia
- theo dõi trạng thái và retry nếu cần
- tổng hợp đầu ra cuối cùng
- biết khi nào cần human review

Worker

- xử lý một năng lực hẹp
- nhận input rõ ràng, trả output rõ ràng
- càng stateless càng dễ test
- thất bại cục bộ không làm hỏng cả kiến trúc
- có thể thay thế mà không ảnh hưởng supervisor

Supervisor giữ **decision flow**; worker giữ **domain skill**. Đừng để một worker vừa làm việc hẹp vừa bí mật điều phối cả hệ thống.

Một worker nên có một năng lực chính: retrieve, gọi tool, tóm tắt, kiểm tra policy...

Nếu có thể, worker chỉ cần input hiện tại thay vì ôm cả lịch sử hệ thống.

Có input / output rõ để test độc lập trước khi cắm vào supervisor.

Lưu ý: Worker mơ hồ thường làm debugging cực khó: không biết lỗi do route sai, prompt sai, hay contract đầu vào chưa đủ rõ.

God Supervisor

Supervisor làm quá nhiều: plan, retrieve, synthesize, monitor. Nó trở thành single-agent được đổi tên.

Implicit State

State bị truyền qua biến toàn cục hoặc side effect. Không ai biết hệ đang ở bước nào.

Chatty Workers

Workers liên tục gọi ngược lại supervisor để hỏi thêm. Message overhead tăng nhanh.

No Fallback

Worker gặp lỗi và không trả về gì. Supervisor chờ mãi không thấy đầu ra để tổng hợp.

Shared state

- dễ xem toàn cảnh
- tiện cho graph orchestration (LangGraph)
- nhưng dễ bị “đụng tay” nếu không có kỷ luật
- cần schema rõ: ai đọc gì, ai ghi gì

Message passing

- contract rõ hơn giữa các agents
- ít coupling hơn
- nhưng phải thiết kế message format cẩn thận
- cần validation ở mỗi điểm nhận

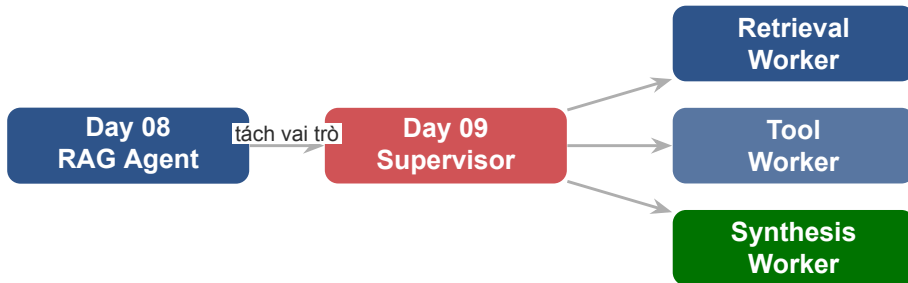
Trong Day 09: **shared state giúp điều phối**, còn **message contract giúp giao tiếp không nhập nhằng**. Day 09 dùng shared state (hợp với LangGraph) làm trực chính.

Những trường cần có trong shared state

- task: task gốc từ user
- plan: danh sách worker cần gọi
- worker_results: output từng worker
- status: pending | running | done | error
- final_answer: tổng hợp cuối
- trace: log dạng list có timestamp

Vì sao trace là bắt buộc?

Không có trace trong state, sau khi hệ chạy xong không ai biết agent đã đi con đường nào để ra kết quả đó.



Day 09 không bắt đầu từ số 0. Ta lấy năng lực retrieve và answer của Day 08 rồi **chia vai trò thành các worker** để hệ thống rõ ràng hơn.

MCP – Model Context Protocol

Nếu supervisor-worker là cách chia người làm việc, thì MCP là cách agent cắm được vào năng lực bên ngoài mà không phải custom từng tích hợp từ đầu

Trước MCP – vấn đề thực tế

- mỗi tool cần một adapter riêng
- đổi API của tool = viết lại code tích hợp
- mỗi framework gọi tool một kiểu khác nhau
- không có chuẩn chung để agent biết tool làm gì

MCP giải quyết

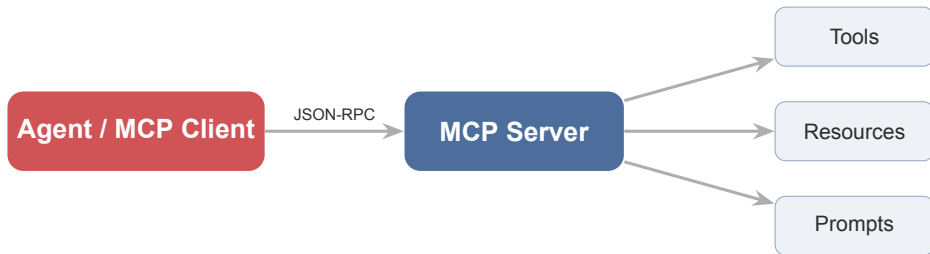
Một chuẩn giao tiếp duy nhất giữa agent và tool. Agent biết cách “khám phá” capability mà không cần hard-code từng tích hợp.

Lưu ý: Điều quan trọng với người học là hiểu **vì sao MCP giúp mở rộng hệ thống**, không phải học thuộc protocol spec trong buổi đầu tiên.

- **MCP** là một chuẩn (Anthropic, mở mã 11/2024) để agent kết nối với external capabilities.
- Thay vì mỗi tool một kiểu tích hợp, agent nói chuyện với một **MCP server** qua **JSON-RPC**.
- MCP server công bố **tools**, **resources**, và **prompts**.
- Agent có thể list, describe, và invoke các capability đó theo chuẩn chung.

Analogy dễ nhớ

Supervisor-worker nói về **vai trò**.
MCP nói về **ổ cắm chuẩn** để agent dùng tài nguyên bên ngoài – giống **USB**: mọi thiết bị cùng một chuẩn kết nối.



Mỗi host chạy nhiều client, mỗi client nối **1:1** tới một server. Agent không cần biết chi tiết hệ thống phía sau – nó chỉ cần hiểu **MCP surface** mà server công bố.

Hành động / thao tác.

Ví dụ: search, query API, tạo ticket, gọi webhook

Tài nguyên để đọc.

Ví dụ: file, schema, catalog, config, DB

Prompt dùng lại.

Giúp chuẩn hóa cách gọi năng lực và giảm lỗi prompt

Lưu ý: Không phải MCP server nào cũng có đủ cả ba. Điều quan trọng là agent có một **cách nhìn nhất quán** về các capability được công bố.

stdio

JSON-RPC qua stdin/stdout cho server chạy **local** (subprocess). Client nên hỗ trợ khi có thể.

Streamable HTTP

Một endpoint (POST + GET), tùy chọn streaming. Cho server **remote**; phải validate Origin để chống DNS-rebinding.

Lưu ý: Lưu ý 2026: Streamable HTTP (3/2025) đã **thay thế** transport HTTP+SSE cũ. SSE giờ chỉ là fallback deprecated – **không phải** một transport thứ ba còn dùng.

Luồng discovery

1. Agent kết nối tới MCP server
2. Gọi `tools/list` để lấy danh sách tool
3. Mỗi tool trả về: `name`, `description`, `inputSchema`
4. Agent đọc schema, chọn tool phù hợp
5. Gọi `tools/call` với đúng parameters
6. Server thực thi và trả kết quả

Tại sao quan trọng?

Agent không cần được lập trình sẵn biết tool “X” tồn tại. Nó có thể **khám phá khi chạy** và tự điều chỉnh theo tool nào có sẵn.

Kịch bản: Customer Support Agent

- Tool Worker cần tra policy mới nhất
- Gọi MCP server “knowledge-base”
- Server expose tool `search_policy`, `get_faq`
- Worker gọi `search_policy(query, date_after)`
- Kết quả JSON chuẩn kèm source

Lợi ích trực tiếp

- team đổi policy → chỉ update MCP server
- supervisor/worker không cần sửa khi tool đổi
- thêm tool mới = thêm endpoint vào server
- dễ audit ai gọi tool gì, khi nào

Hội tụ cross-vendor (2025)

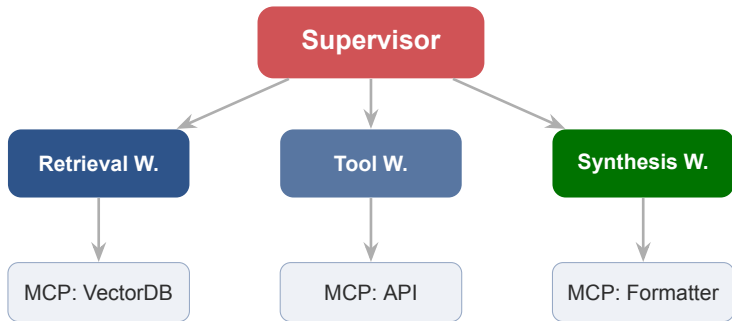
- **OpenAI**: MCP trong Agents SDK + Responses API
- **Google**: tuyên bố hỗ trợ Gemini (4/2025)
- **Microsoft**: GA trong Copilot Studio

Hội tụ là thật; nhưng độ sâu hỗ trợ mỗi bên khác nhau.

Security cần biết

- OAuth 2.1 (draft) + token audience-bound
- **tool poisoning**: mô tả tool độc hại có thể lái agent
- **prompt injection** từ tool output

Mitigation là client-side: coi output là untrusted, duyệt tool rủi ro, sandbox.



MCP là **lớp giữa** worker và capability thật. Worker chỉ cần biết MCP surface, không cần biết hệ phía sau là gì – đó là **ecosystem effect**.

A2A – Agent to Agent Communication

MCP giúp agent nói chuyện với tool; A2A giúp agent nói chuyện với agent khác theo cách rõ nhiệm vụ, rõ bối cảnh, và rõ đầu ra mong đợi

MCP – agent ↔ tool

- agent nói chuyện với **tool / capability**
- mục tiêu: kết nối năng lực bên ngoài
- tool không có agency – chỉ thực thi

A2A – agent ↔ agent

- agent nói chuyện với **agent khác**
- mục tiêu: chia việc và đồng bộ
- agent phía kia **có thể ra quyết định**

MCP trả lời “agent lấy năng lực ở đâu?” (chiều dọc, ra ngoài). **A2A** trả lời “agent giao việc cho ai?” (chiều ngang, giữa các agent). Hai thứ **bổ sung**, không cạnh tranh.



- Donate cho **Linux Foundation** (vendor-neutral); founding members: AWS, Cisco, Google, Microsoft, Salesforce, SAP, ServiceNow
- IBM **ACP** (Agent Communication Protocol) đã gộp vào A2A (8/2025) – hội tụ về một chuẩn agent-to-agent

Lưu ý: Phân biệt 2 mốc: A2A được **công bố** 4/2025, nhưng **donate** cho Linux Foundation 6/2025. Con số adoption (150+ công ty) là ước lượng từ blog, không phải spec.

Agent Card

Một JSON metadata agent công bố để quảng bá: identity, capabilities, **skills** (tác vụ cụ thể agent làm được), endpoint, auth. Một client agent đọc card để biết **ai làm được task này**.

Đường dẫn quy ước:
`/.well-known/agent-card.json`

Core objects

- Task: đơn vị công việc có lifecycle
- Message: một lượt trao đổi (user/agent)
- Part: text / file / structured data
- Artifact: kết quả tạo ra

A2A xây trên web standards (HTTP, JSON-RPC, SSE). Agent là **peer hộp đen** (opaque): cộng tác mà **không chia sẻ memory / tool** nội bộ của nhau.

Agent kia cần làm gì?

Ví dụ: tìm 3 chunk policy
phù hợp nhất

Worker cần biết gì để
làm đúng?

query, constraints, user
role, state

Trả về theo format nào?
list chunks, score, ratio-
nale, error

Ví dụ:

task = "retrieve evidence"

context = "user hỏi về refund policy, ưu tiên tài liệu sau 2025"

expected_output = "top 3 chunks + source + confidence"

Thiếu context

worker lấy kết quả không phù hợp · phải gọi lại nhiều lần · debugging rất khó

Quá nhiều context

tốn token · worker xử lý chậm · khó maintain khi schema đổi

Nguyên tắc “need to know”

Worker chỉ nhận context nó **thực sự cần** để hoàn thành task. Không thêm, không bớt.

Khi không chắc → bắt đầu với **ít hơn** và thêm khi cần.

Sync

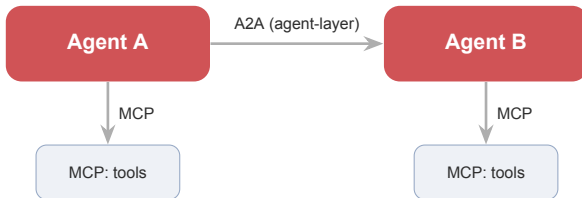
- đơn giản để hiểu và debug
- hợp khi supervisor cần kết quả ngay
- nhưng dễ tăng latency toàn luồng
- **bắt đầu ở đây cho Day 09**

Async

- hợp khi nhiều worker chạy song song
- tận dụng được concurrency
- nhưng cần quản lý trạng thái và timeout tốt hơn
- mở rộng khi đã nắm sync tốt

Sync dễ dạy cho vòng đầu. Async chỉ cần giới thiệu như một hướng mở rộng khi nhiều worker có thể chạy độc lập.

- ☒ **Biết rõ ai được gọi ai:** không phải agent nào cũng được chạm mọi capability
- ☒ **Biết dữ liệu nào được truyền đi:** tránh đẩy thừa PII hoặc state nhạy cảm
- ☒ **Biết output nào cần xác thực lại:** đặc biệt khi worker chạm tool ngoài
- ☒ **Validate đầu ra trước khi dùng:** worker có thể trả về schema sai
- ☐ **Đừng giả định mọi agent đều đáng tin như nhau.** Hệ nhiều agent vẫn cần trust boundary



Một app dùng **A2A** để nói chuyện với agent khác; mỗi agent dùng **MCP** để gọi tool của riêng mình. ACP (IBM) đã gộp vào A2A – hệ sinh thái đang hội tụ về 2 lớp này.

Orchestration Với LangGraph

Sau khi hiểu vai trò và giao tiếp, ta cần một cách biểu diễn luồng chạy rõ ràng; LangGraph là cách rất trực quan để làm điều đó

Không có framework

- routing logic nằm trong một prompt điều phối dài
- khó biết hệ đang ở bước nào
- thêm một nhánh mới = sửa cả prompt
- debug bằng `print()` và hy vọng

Với LangGraph

Routing trở thành **code tường minh**. Graph visualize được, state có schema, human-in-the-loop có điểm rõ ràng.

LangGraph **1.0 GA** (10/2025). Các primitive lõi (StateGraph, nodes, edges) ổn định; lab sẽ pin version để code chạy đúng.

Node

- là một hàm Python nhận state, trả về phần state cần cập nhật
- tương ứng một agent / một bước (supervisor, worker, human review)

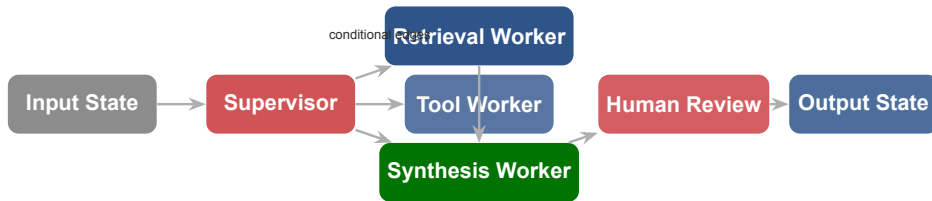
Edge

- **unconditional**: luôn đi $A \rightarrow B$
- **conditional**: hàm trả tên node tiếp theo dựa trên state

State

TypedDict (hoặc Pydantic) truyền qua mỗi node. Mỗi node đọc toàn bộ state, ghi vào các trường được phép.

node = ai làm · edge = đi đâu · state = hệ biết gì



LangGraph làm rõ **route** quyết định ở đâu, **state** đi qua đâu, và **human** can thiệp ở điểm nào.

```
from langgraph.graph import StateGraph, START
from typing import TypedDict

class AgentState(TypedDict):
    task: str
    need_tool: bool
    need_retrieval: bool
    trace: list

def route(state) -> str:      # supervisor's decision
    if state["need_tool"]:    return "tool_worker"
    if state["need_retrieval"]: return "retrieval_worker"
    return "synthesis_worker"

g = StateGraph(AgentState)
g.add_node("supervisor", supervisor_node)  # + worker nodes: lab
g.add_edge(START, "supervisor")
g.add_conditional_edges("supervisor", route)
app = g.compile()
```

Routing trở thành logic tường minh, thay vì ẩn trong một prompt điều phối dài. (Các worker node được add ở lab.)

```
def supervisor_node(state: AgentState) -> AgentState:
    decision = llm.invoke(
        SUPERVISOR_PROMPT.format(task=state["task"])
    )
    return {
        **state,
        "need_retrieval": decision.need_retrieval,
        "need_tool": decision.need_tool,
        "trace": state["trace"] + [
            f"[supervisor] tool={decision.need_tool}"
        ],
    }
```

Supervisor node **không làm việc của worker** – chỉ ra quyết định route và **ghi trace**. Mọi trường nó dùng đều có trong AgentState.


```
from langgraph.checkpoint.memory import InMemorySaver

# compile WITH a checkpointer -> enables memory + resume
app = g.compile(checkpointer=InMemorySaver())

config = {"configurable": {"thread_id": "session-1"}}
app.invoke(state, config)  # same thread_id => same conversation
```

- Checkpointer cho phép **multi-turn memory** và **resume sau interrupt** – cùng thread_id = cùng cuộc.
- InMemorySaver cho lab; production dùng SqliteSaver / PostgresSaver (cài gói riêng).

```
from langgraph.types import interrupt, Command

def human_review(state: AgentState):
    # pause the graph; a human inspects, then we resume
    interrupt({"last_step": state["trace"][-1]})
    return {"trace": state["trace"] + ["[human] approved"]}

# resume the SAME thread (app compiled with checkpointers above):
config = {"configurable": {"thread_id": "session-1"}}
app.invoke(Command(resume="approve"), config)
```

Idiom 2026: `interrupt()` động trong node + `Command(resume=)` – **cần checkpointer** (frame trước). `interrupt_before` tính chỉ còn để debug. Node **chạy lại từ đầu** khi resume – giữ side-effect idempotent.

Agent Teams

Worker có **context riêng**, chỉ **report kết quả** về agent chính – không nói chuyện với nhau.

≈ **supervisor-worker**

Nhiều phiên Claude **ngang hàng**, chia **task list chung**, **nhắn trực tiếp** cho nhau.

≈ **A2A** (agent ↔ agent)

Một **script** giữ kế hoạch (loop, nhánh, kết quả) điều phối **hàng chục-trăm agent**.

≈ **orchestration** (như Lang-Graph)

Subagents & Teams: **Claude/lead quyết từng lượt**; Workflow: **script quyết**. Nơi giữ kết quả trung gian: context → task list → biến trong script. Cùng nguyên lý Day 09 – chia vai trò, giao tiếp rõ, điều phối tường minh – đang chạy trong sản phẩm thật.

Nguồn: code.claude.com/docs/en/agent-teams & [/workflows](https://code.claude.com/docs/en/workflows). Agent teams = experimental; dynamic workflows = research preview (2026).

Observability & Debugging Hệ Multi-Agent

Hệ nhiều agent khó debug hơn nhiều so với single agent; observability tốt là điều kiện tiên quyết để cải thiện được hệ thống

Nguồn gốc lỗi khó xác định

- lỗi có thể từ: routing sai, context sai, tool fail, synthesis sai
- lỗi ở bước A có thể chỉ lộ ra ở bước C
- nhiều agent = nhiều LLM call = nhiều điểm fail tiềm năng

3 câu hỏi observability

1. Agent nào đã chạy, theo thứ tự nào?
2. Input/output tại mỗi bước là gì?
3. Lỗi hay warning nào đã xảy ra?

Lưu ý: Không có trace tốt, debugging multi-agent gần như là mò mẫm. Đặc biệt: lỗi thường **ngũ nghĩa** (agent hiểu sai output) chứ không phải crash – trace phải ghi I/O thật, không chỉ latency.

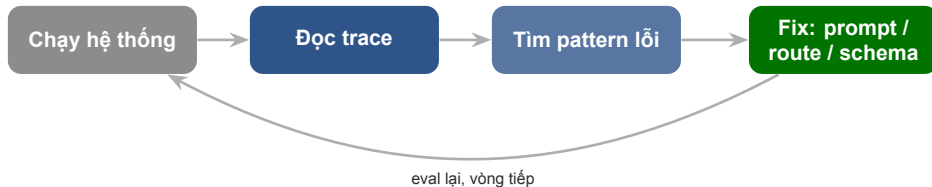
Mỗi entry nên có

- timestamp: khi nào
- agent_id: ai làm
- action: route / call / synth / error
- input_summary / output_summary
- status: ok | warn | error
- latency_ms: mất bao lâu

```
{  
  "t": "14:03:21",  
  "agent": "supervisor",  
  "action": "route",  
  "decision": "retrieval_worker",  
  "status": "ok",  
  "latency_ms": 312  
}
```

- supervisor nhận task gì và route theo tiêu chí nào
- worker nào được gọi và input nó nhận là gì
- worker trả về output gì, confidence hoặc status gì
- supervisor đã tổng hợp ra answer cuối cùng như thế nào
- điểm nào bị retry, timeout, hoặc fallback
- nếu có human review, human đã quyết định gì

Lưu ý: Nếu log chỉ có “agent chạy xong”, học viên sẽ không học được gì về orchestration. Trace tốt phải giúp nhìn thấy **đường đi của quyết định**.



Trace không chỉ để debug lỗi hôm nay. Nó là **dữ liệu để cải thiện** routing, worker quality, và message contract theo thời gian.

Span cho agentic systems

- `invoke_agent`, `execute_tool`,
`invoke_workflow`
- `attribute gen_ai.*`: `agent.name`,
`request.model`, `input/output.messages`
- `cost`: `usage.input_tokens`,
`usage.output_tokens`

Lưu ý (2026)

OpenTelemetry GenAI semconv vẫn ở trạng thái **Development** (chưa stable). Dạy như **hướng hội tụ** của hệ sinh thái, không phải spec đóng băng.

JSON trace tự viết của bạn **chính là** một cây span (agent, tool, I/O, latency, status). OTel chỉ là cách chuẩn hóa nó.

Của LangChain, framework-agnostic.

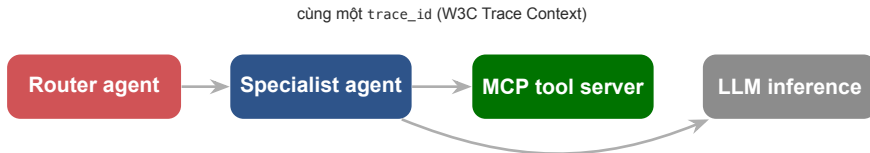
Managed-first (không có bản OSS self-host miễn phí).

Open-source (MIT), self-host được. Tốt cho data sovereignty.

Open-source, OTel-native. Thiên về **local dev / debug**.

Apache 2.0, OTel extensions (Traceloop). Export sang 25+ back-end.

Bắt đầu bằng **JSON log tự viết vào state** – học được nhiều nhất về cách hệ hoạt động. Sau đó “graduate” sang một tool để có search, latency stats, và cost dashboard miễn phí.



Một request đi qua nhiều service. Cùng một `trace_id` truyền qua header giữ các span **nối lại thành một waterfall** – MCP server cũng là một “peer service” có span riêng.

Cost, Latency & Reliability

Multi-agent không miễn phí; nhiều agent = nhiều LLM call = nhiều tiền và nhiều điểm fail. Cần tư duy trade-off từ sớm

09

Chi phí tăng từ đâu?

- mỗi agent = ít nhất 1 LLM call
- supervisor thường là một LLM call riêng
- context truyền qua state có thể lớn
- retry = gấp đôi cost tại điểm đó

Nguyên tắc tối ưu

Supervisor **không cần** model lớn nhất. Nếu routing đơn giản, dùng model nhỏ cho supervisor và giữ model mạnh cho worker cần reasoning sâu.

~4×

token của một **agent** so với một lần chat

~15×

token của **multi-agent** so với một lần chat

~80%

phần variance hiệu năng (trên BrowseComp) chỉ riêng **token usage** giải thích

Lưu ý: Phần lớn “multi-agent thắng” thực ra là “**tiêu nhiều compute hơn**”. Chỉ đáng cho task giá trị cao. (*Lưu ý: đây là số tự báo cáo của Anthropic, không phải benchmark độc lập.*)

Tiêu chí	Single Agent	Multi-Agent
Chi phí LLM	Thấp hơn (1 call)	Cao hơn (nhiều call)
Latency	Thấp nếu prompt ngắn	Có thể song song; tăng nếu serial
Debuggability	Khó hơn (logic ẩn)	Tốt hơn (có trace rõ)
Specialization	Hạn chế	Tốt hơn (mỗi worker chuyên biệt)
Scalability	Khó scale vai trò	Dễ thêm worker mới
Complexity	Đơn giản hơn	Phức tạp hơn khi setup

Trade-off quan trọng cần hiểu khi thiết kế

Cần thiết kế trước

- worker có **timeout** rõ ràng
- supervisor có **retry logic**: thử lại bao nhiêu lần?
- nếu retry cũng fail: **fallback** là gì?
- **partial failure**: tổng hợp với những worker đã thành công

Graceful degradation

Hệ không cần làm tốt mọi trường hợp. Nó cần **thất bại có kiểm soát** và báo cáo rõ ràng khi không đủ tự tin.

Lưu ý: Thất bại cục bộ của một worker **không nên** crash toàn bộ hệ thống – đó là lợi thế chính của việc chia vai trò.

Kế Hoạch Học Tập Ngày 9

Roadmap rõ ràng giúp học viên biết nên đầu tư thời gian vào đâu và cần nắm vững điều gì trước khi chuyển sang bước tiếp

10

- **Phần 1 (60 phút)** – Lý thuyết: single-agent limits, mental model, 4 patterns
- **Phần 2 (45 phút)** – Supervisor-worker deep dive + shared state + anti-patterns
- **Phần 3 (45 phút)** – MCP architecture + A2A message contract
- **Phần 4 (30 phút)** – LangGraph: nodes, edges, state, routing code
- **Phần 5 (30 phút)** – Observability + cost/reliability trade-offs
- **Lab (90 phút)** – Nâng cấp artifact Day 08 thành multi-agent + MCP

Hands-on trước, docs sau. Đọc code example thực tế trước khi đọc architecture spec đầy đủ.

Từ Day 08 (bắt buộc)

- RAG pipeline: query → retrieve → generate
- tool calling cơ bản
- cách viết system prompt có cấu trúc
- artifact Day 08 đang hoạt động

Python foundations

- TypedDict và type hints
- dictionary operations
- function decorators cơ bản
- async/await (nếu đi vào async A2A)

Lưu ý: Học viên chưa có artifact Day 08 hoạt động nên dành 30 phút đầu fix artifact trước khi bắt đầu lab Day 09.

Cấp 1: Foundation

- giải thích 4 giới hạn single-agent
- vẽ sơ đồ supervisor-worker
- phân biệt MCP và A2A

Cấp 2: Implementation

- build supervisor + 2 workers với shared state
- viết message contract cho A2A
- dùng MCP cho 1 tool worker

Cấp 3: Mastery

- LangGraph có conditional routing
- trace log đọc được và actionable
- giải thích trade-off cost/reliability

Misconception	Reality
“Nhiều agent = hệ thống tốt hơn”	Nhiều agent = nhiều phức tạp; chỉ nên dùng khi cần
“Supervisor phải là model lớn nhất”	Supervisor chỉ cần đủ để route đúng
“MCP và A2A là cùng một thứ”	MCP: tool integration; A2A: agent delegation
“Multi-agent tự động giải quyết context problem”	Context vẫn phải được quản lý cẩn thận
“LangGraph chỉ dùng được với LangChain”	LangGraph dùng được với nhiều LLM framework

Sửa sớm để học viên không mang theo hiểu lầm

Đọc trước (chuẩn bị)

- LangGraph Quickstart (docs.langchain.com)
- MCP Introduction – modelcontextprotocol.io
- Sumers et al. (2023), CoALA – Section 2–3
- Review lại artifact Day 08 của bản thân

Đọc sau (củng cố)

- Anthropic – Building Effective Agents
- Anthropic – multi-agent research system (2025)
- A2A Protocol – a2a-protocol.org
- LangSmith Tracing Guide

Đọc code example thực tế trước, architecture spec đầy đủ sau.

Hands-on 9

Lấy artifact Day 08 rồi chia lại thành supervisor, workers, và 1 điểm kết nối MCP để học viên thấy rõ giá trị của phân vai trong thực tế



Biến một agent RAG đơn thành một hệ multi-agent nhỏ có route rõ, capability rõ, và trace rõ để dễ giải thích và debug hơn.

1. tách artifact Day 08 thành supervisor + 2–3 workers
2. thiết kế shared state schema với trường trace
3. chọn 1 worker dùng external capability qua MCP
4. viết message contract tối thiểu giữa supervisor và workers
5. trace lại toàn bộ luồng để biết agent nào đã làm gì
6. demo kết quả cuối cùng kèm reasoning flow ở mức quan sát được

Câu hỏi cần trả lời

- RAG agent Day 08 đang làm bao nhiêu việc khác nhau?
- phần nào có thể tách thành worker riêng?
- phần nào nên để supervisor quyết định?

Gợi ý tách vai trò

- **Retrieval W.:** vector search + rerank
- **Tool W.:** gọi API qua MCP
- **Synthesis W.:** generate final answer

Kiểm tra trước khi tách

Mỗi phần có thể **test độc lập** không? Nếu không, chưa tách đủ rõ. Supervisor có thực sự cần ra quyết định về phần đó không? Nếu không, tách ra là dư thừa.

```
from typing import TypedDict, Optional
```

```
class Day09State(TypedDict):  
    task: str  
    user_context: dict  
    plan: list[str]  
    retrieval_result: list[dict]  
    tool_result: dict  
    synthesis_draft: str  
    final_answer: str  
    status: str  
    trace: list[dict]  
    error: Optional[str]
```

Nguyên tắc thiết kế

- trace là list, **append** không overwrite
- error là Optional để graceful fail
- worker chỉ ghi vào field của mình
- supervisor đọc tất cả, ghi plan

Chọn một lựa chọn lab

- **A:** dùng MCP server demo có sẵn (search / weather)
- **B:** tự viết một MCP server đơn giản với 1 tool
- **C:** mock MCP interface với real HTTP call

Điều cần chứng minh

- Tool Worker gọi MCP để lấy dữ liệu
- kết quả từ MCP được ghi vào shared state
- trace ghi lại lần gọi MCP

Điều quan trọng nhất

Không quan trọng tool làm gì. Quan trọng là học viên thấy được cách agent **kết nối với capability bên ngoài theo chuẩn**, không phải hard-code.

System pieces

- 1 supervisor với routing logic rõ
- 2–3 workers rõ vai trò
- 1 MCP-connected capability
- shared state schema có trace field

Evidence

- trace / logs đọc được
- output cuối cùng với source
- ghi chú: route hợp lý chưa? tại sao?
- ≥ 1 ví dụ worker fail gracefully

Lưu ý: Không cần build hệ enterprise. Điều cần chứng minh là **việc chia vai trò giúp hệ thống rõ hơn, dễ kiểm soát hơn, và mở rộng tốt hơn.**

Tiêu chí	Đạt (70%)	Tốt (85%)	Xuất sắc (100%)
Phân vai trò	supervisor + 2 workers	vai rõ, không overlap	anti-patterns được tránh có ý thức
MCP kết nối	1 MCP call hoạt động	schema rõ, trace ghi được	discovery đúng, có error handling
Shared state	state hoạt động	schema đầy đủ, có trace field	ownership rõ từng field
Trace quality	log cơ bản	đủ các trường cần thiết	actionable, dẫn đến insight
Routing logic	routing hoạt động	conditional edge rõ	giải thích được quyết định route

Lab 9 – nộp để instructor đánh giá

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 **Multi-agent** chia vai trò để hệ đỡ quá tải, dễ kiểm soát – không phải chỉ tăng số agent. Chỉ dùng khi cần.
- 2 **Supervisor-worker** là pattern thực dụng nhất để bắt đầu: supervisor route và tổng hợp; worker chuyên, stateless, testable.
- 3 **MCP** nối agent với tool; **A2A** cho agents giao việc cho nhau – hai vấn đề khác nhau.
- 4 **LangGraph** biến orchestration thành graph có state và conditional routing – dễ debug và thêm human-in-the-loop.
- 5 **Observability**: trace tốt không chỉ để debug hôm nay mà là dữ liệu để cải thiện hệ thống.

Agent UX & Thiết Kế Trải Nghiệm AI

“Hệ thống đã thông minh hơn và phối hợp tốt hơn. Nhưng nếu trải nghiệm kém, user vẫn sẽ không muốn dùng.”

- Nếu supervisor và workers làm đúng nhưng user không hiểu chuyện gì vừa xảy ra, trải nghiệm có đủ tốt không?
- Quan sát lab hôm nay để nhận ra chỗ nào cần transparency, confidence indicator, và human handoff
- Chuẩn bị 1 use case để mô tả AI flow từ góc nhìn người dùng thay vì chỉ từ kiến trúc

1. Model Context Protocol, *Official Documentation & Spec* – modelcontextprotocol.io (architecture, transports, authorization).
2. Google / Linux Foundation, *Agent2Agent (A2A) Protocol* – a2a-protocol.org.
3. Anthropic (2024), *Building Effective Agents*; Anthropic (2025), *How we built our multi-agent research system*.
4. LangGraph Docs, *Multi-Agent & Human-in-the-Loop* – docs.langchain.com/oss/python/langgraph.
5. OpenTelemetry, *GenAI Semantic Conventions*; Summers et al. (2023), *CoALA*.

Hỏi & Đáp

Một agent rất giỏi có thể làm nhiều việc. Nhưng hệ thống tốt hơn thường bắt đầu từ câu hỏi: nên chia vai trò ở đâu để dễ kiểm soát và dễ cải thiện nhất?